OFFICIAL RELEASE GUIDE v1.0.0

WPT-PRO & WPT-PRO Plus

Automated, Multi-Stage Windows System Optimization & Servicing Engine

WPT-PRO is an enterprise-grade PowerShell automation suite designed to maintain Windows operating systems at peak performance. By combining deep kernel tuning, working set memory trimming, component store cleanup (DISM/SFC), and automated reboot handoffs, WPT-PRO consistently reduces system process footprints down to **~67 processes** and settles idle CPU usage at **0%–1%**.

KEY FEATURES & CAPABILITIES

Autonomous Execution

Executes across two complete system reboots via scheduled transit bridges without requiring active human intervention.

Dual Operating Modes

Features a 13-minute Daily Standard Pass and a 2-3 hour Deep Weekend Plus Pass for thorough servicing.

Cornerstone Duration Tracking

Anchors total wall-clock metrics strictly to TOM's initial baseline timestamp, eliminating timer drift across reboots.

Smart Memory & Kernel Tuning

Includes dynamic working set trimming and an 18-step deep OS kernel optimization pass.

PIPELINE HANDOFF ARCHITECTURE



- **TOM (Pre-Flight & Baseline):** Captures system KPIs, writes baseline anchor to `Pre_Opt_KPI.json`, purges browser caches, and registers reboot handoff.
- **ROBERT_v0 (Core Engine):** Runs NGEN assembly compilation, executes 18-step deep kernel tuning, and flushes process working sets.
- **FinKPI (Post-Servicing):** Captures final post-optimization metrics, calculates wall-clock duration against TOM baseline, and generates `performance.html`.

OPERATIONAL MODES COMPARISON

Feature / Stage	Standard Pass (wpt-pro.ps1)	Weekend Plus Pass (wpt-plus.ps1)
Target Runtime	~12 – 14 Minutes	~2 – 3 Hours
Execution Frequency	Daily (Monday – Friday)	Weekly (Weekend Maintenance)
Reboot Cycles	2 Automated Reboots	2 Automated Reboots
DISM / SFC Cleanup	Rapid Component Pass	Full Deep Component Store Cleanup
NGEN Compilation	Core Framework Queue Only	Comprehensive Assembly Compilation
HTML Report Trigger	Immediate upon Phase 3 Exit	Deferred until all servicing finishes

SYSTEM REQUIREMENTS

- **Operating System:** Windows 10 / Windows 11 (64-bit) or Windows Server 2019+
- **PowerShell Core:** PowerShell 7.x (`pwsh.exe`) installed at `C:\WPT\pwsh\pwsh.exe`
- **Privileges:** Administrator / SYSTEM elevation (required for registry modification, DISM, and reboot tasks)
- **Execution Policy:** Configured to `Bypass` or `Unrestricted`

DIRECTORY STRUCTURE

```
C:\WPT\ ├── pwsh\ # Portable PowerShell 7 Core Binaries | └── pwsh.exe ├── script
s\ # Core Pipeline Scripts | ├── TOM_v0.ps1 # Phase 1: Pre-flight & Baseline Capture |
├── ROBERT_v0.ps1 # Phase 2: Kernel Optimization & NGEN | ├── FinKPI_v0.ps1 # Phase 3:
Analytics & HTML Generation | ├── wpt-pro.ps1 # Daily Entry Point | └── wpt-plus.ps1 #
Weekend Plus Entry Point ├── bridge\ # Transit Handoff Scripts ├── DATA\ # Persistent J
SON State (Pre_Opt_KPI.json) ├── Reports\ # Generated Diagnostics (performance.html) └
─ Logs\ # Unified Pipeline Execution Logs
```

INSTALLATION & SETUP

1. **Extract Package:** Place the complete `WPT` directory at the root of your primary system drive (`C:\WPT\`).
2. **Set Execution Policy:** Run an elevated PowerShell prompt:

   ```
   Set-ExecutionPolicy -ExecutionPolicy Bypass -Scope LocalMachine -Force
   ```

3. **Verify Task Scheduler:** Ensure administrative rights exist to create scheduled tasks under `\WPTPRO_*`.

USAGE & EXECUTION COMMANDS

Running a Daily Pass (Standard ~13 Min)

```
powershell.exe -ExecutionPolicy Bypass -NoProfile -WindowStyle Minimized -File "C:\WPT\scripts\wpt-pro.ps1"
```

Running a Weekend Plus Pass (Deep Servicing ~2-3 Hours)

```
powershell.exe -ExecutionPolicy Bypass -NoProfile -WindowStyle Minimized -File "C:\WPT\scripts\wpt-plus.ps1"
```

Standalone Diagnostic Reporting Mode

```
C:\WPT\pwsh\pwsh.exe -File "C:\WPT\scripts\FinKPI_v0.ps1"
```

STATE MANAGEMENT & DURATION TRACKING

To eliminate timer drift during multi-reboot sequences or extended servicing passes, WPT-PRO enforces **Cornerstone Epoch Anchoring**:

- **Baseline Timestamp Capture:** TOM captures an explicit ISO 8601 timestamp and writes it directly to `C:\WPT\DATA\Pre_Opt_KPI.json`.
- **Duration Calculation:** `FinKPI_v0.ps1` reads TOM's original timestamp as its primary baseline, calculating duration as:
  `Total Duration = Time(FinKPI Exit) - Time(TOM Baseline Init)`

TROUBLESHOOTING & RUNBOOK

Pipeline Stopped After Reboot #1

Cause: Task Scheduler handoff was blocked or permission denied.
Fix: Check Task Scheduler for `WPTPRO_Phase1To2_Handoff` and review `C:\WPT\Logs\wpt_execution.log`.

Duration Formatting Anomaly

Cause: Script utilizing total minutes instead of hour-based parsing.
Fix: Verify `FinKPI_v0.ps1` uses `[math]::Floor($elapsed.TotalHours)` for formatting.

Manual Pipeline Reset Protocol

If a run is interrupted mid-transit, execute this reset command block in an elevated terminal:

```
Remove-Item "C:\WPT\DATA\Pipeline_State.json" -Force -ErrorAction SilentlyContinue Unregister-ScheduledTask -TaskName "WPTPRO_Phase1To2_Handoff" -Confirm:$false -ErrorAction SilentlyContinue Unregister-ScheduledTask -TaskName "WPTPRO_Phase2To3_Handoff" -C
```